

Video RAG Retrieval Project

<https://github.com/aiyshwary/Video-RAG-Retrieval>

OVERVIEW

A semantic video search system that extracts frames, generates scene captions via ViT-GPT2, and embeds them using OpenCLIP. Queries are matched against a dual FAISS index using cosine similarity, returning the most relevant results and frame references.

IMPACT

Enables semantic search over unstructured video content by leveraging vision-language models and vector search, making scenes retrievable by natural language queries.

TECHNICAL WALKTHROUGH

Implements a simple Video Retrieval-Augmented Generation (RAG) system for semantic video search using video frame embeddings and performing nearest-neighbor lookups.

- i) Extract frames and scene descriptions from videos using ViT-GPT2 image captioning.
- ii) Compute text embeddings with SentenceTransformers and image embeddings with OpenCLIP.
- iii) Store embeddings in FAISS indexes with associated metadata (video name, scene ID, frame number).
- iv) Retrieve relevant scenes or frames for a text query using cosine similarity.

Clone the repository, create a virtual environment, and install dependencies from requirements.txt. GPU acceleration is supported for faster computation.

Process a video and index its scenes: `extract_scenes('path/to/video.mp4', fps=0.5); retriever = VideoR`
`retriever.index_scenes(scenes, video_name='demo').`
Query: `results = retriever.query('a person walking', k=3, use_text=True).`

/Video RAG Retrieval Project ■■■■ README.md ■■■■ requirements.txt ■■■■ src/video_rag/ ■■■■ __in
embeddings.py ■■■■ storage.py ■■■■ retrieval.py ■■■■ utils.py

- i) Scene detection is rudimentary; can be replaced with advanced shot-detection or segmentation logic
- ii) Metadata stored with embeddings can be extended to track file paths, timestamps, etc.
- iii) For production, swap FAISS for a proper vector database such as VDMS, Milvus, or Pinecone.

KEY DETAILS

1. Extracts frames and scene descriptions from videos using open-source models (ViT-GPT2 for captions, OpenCLIP for image embeddings).
2. Computes text embeddings with SentenceTransformers and image embeddings with OpenCLIP.
3. Stores embeddings in FAISS indexes with associated metadata.
4. Retrieves relevant scenes or frames for a text query.
5. Lightweight Python prototype; can be extended to production vector DBs like VDMS, Milvus, or Pinecone.